

SafeDig AI — Complete End-to-End Execution & Code Working Mechanism

UK Underground Utility Dig-Safety Map QA & Validation Platform

Compliance Standard: UK Health and Safety Guidance 47 (HSG47) & CDM 2015

Core Invariant: Zero Escaped Hazards (**SAFE_MODE=True**)

1. High-Level Architectural Flow & System Overview

SafeDig is an automated, AI-augmented safety validation platform engineered to prevent underground utility strikes during civil excavations across the United Kingdom.

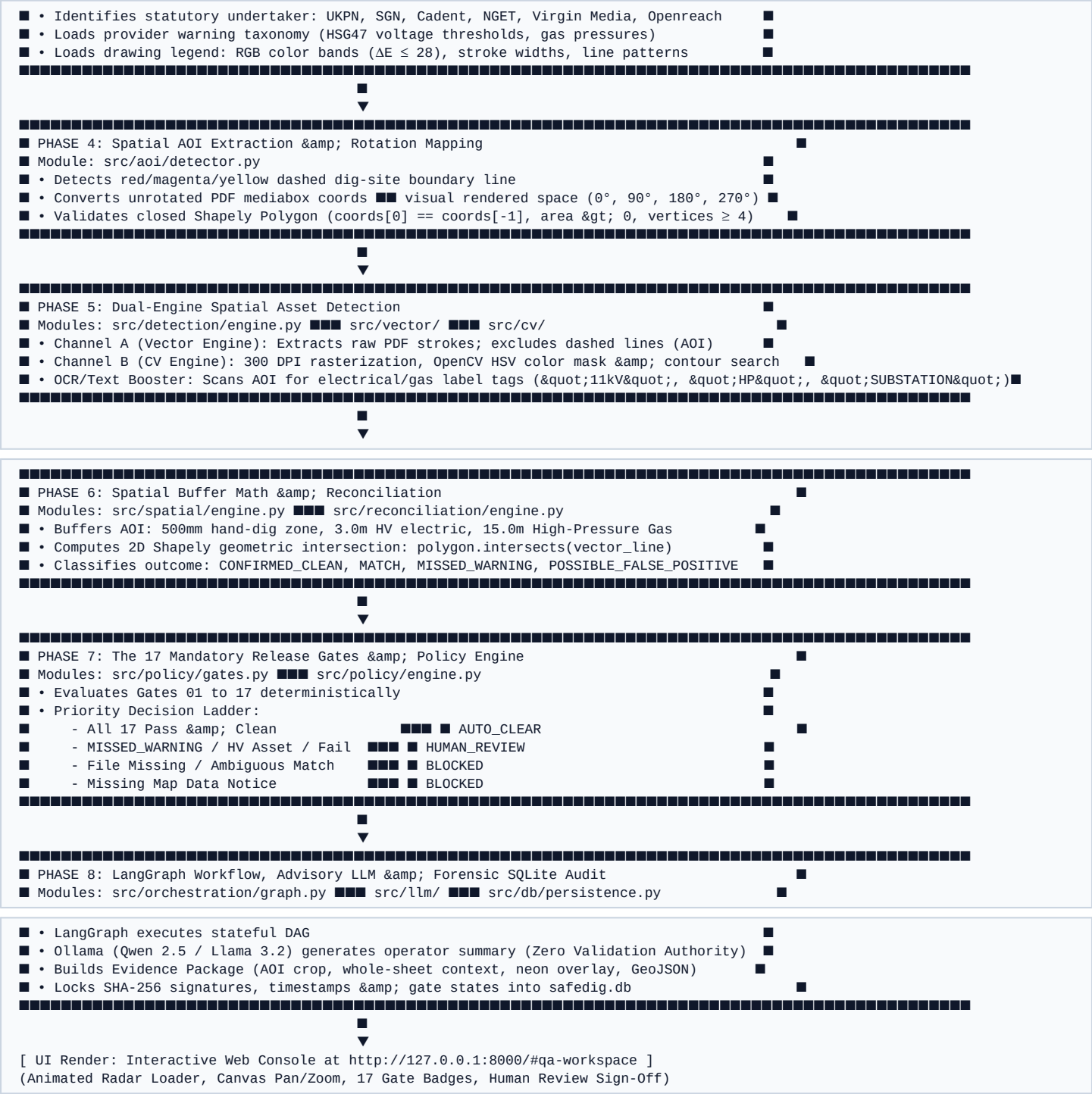
Under UK statutory regulations (**HSG47: Avoiding Danger from Underground Services**), striking buried infrastructure (high-voltage electricity lines, high-pressure gas mains, trunk fiber, or water mains) results in lethal explosions, arc-flash electrocution, service blackouts, and unlimited financial/criminal liability.

SafeDig enforces a non-negotiable architectural invariant:

[IMPORTANT] **Domain Safety Invariant ([SAFE_MODE=True](#)) SafeDig re-validates upstream utility warning claims against actual CAD/GIS engineering drawings within the excavation site. The target is zero escaped hazards. Every detected hazard is escalated to a certified human safety engineer. Every ambiguous, corrupted, contradictory, or degraded case must fail toward [HUMAN_REVIEW](#) or [BLOCKED](#)—**never toward [AUTO_CLEAR](#)**. A false negative is never traded for a cleaner false-positive metric.**

```
[ User Action: Web Console (Upload/Click) OR CLI (run_map_qa_pipeline) ]
```





2. Detailed Technical Walkthrough by Phase

Phase 1: Intake & Self-Healing Path Resolution

When a job directory is supplied to SafeDig, it contains an `index.xlsx` sheet and an arbitrary collection of PDF files received from utility undertakers.

- 1. ****Self-Healing Path Resolution (`src/utls/paths.py`)**:**
 - Inquiry packs generated on one machine frequently embed foreign absolute file paths (e.g., Linux `/home/runner/work/.../...` or Windows `D:\RawJobs\...`).

- **SafePathResolver** normalizes paths dynamically. It checks whether the target file exists locally. If not, it tokenizes the filename and searches recursively within the job root, healing broken references automatically.
- 2. ****Master Index Ingestion (src/index/parser.py)****:
 - Reads **index.xlsx** via **pandas** and **openpyxl**.
 - Normalizes utility names (e.g. mapping "UK Power Networks plc" to canonical **UKPN**).
 - Parses declared status ("Yes" vs "No"), enquiry ID, boundary description, and raw provider comments.
- 3. ****Inventory Matching & Noise Separation (src/ingestion/inventory.py)****:
 - Statutory utility disclosure packs contain significant "chaff"—cover letters, standard terms, health and safety booklets, hospital plans, and valve operating notices.
 - The inventory matcher uses regex tokenization and fuzzy similarity scoring to separate authoritative CAD utility drawings from non-relevant documents.
 - Maps each **IndexRecord** to its corresponding PDF with a **DocumentResolutionStatus** (**UNIQUE**, **EXCLUDED**, **AMBIGUOUS**, or **NOT_FOUND**).

Phase 2: PDF Document Ingestion & Modality Classification

1. ****Document Integrity Verification (src/pdf/extractor.py)****:
 - Opens the PDF file stream via PyMuPDF (**pymupdf.open(file_path)**).
 - Verifies the **%PDF-** file header signature and calculates the SHA-256 cryptographic digest of the raw bytes.
2. ****Modality Inspection (src/pdf/inspector.py)****:
 - Evaluates page count and inspects the drawing primitives on the map page.
 - **Vector Drawing**: Inspects **page.get_drawings()**. If mathematical line segments, beziers, and polygon paths are found with stroke colors, the document is classified as **PDFModality.VECTOR**.
 - **Raster Drawing**: If the page consists of an embedded bitmap image (e.g. scanned legacy drawing), it is classified as **PDFModality.RASTER**.
 - **Hybrid**: Contains both vector paths and underlying raster imagery (**PDFModality.HYBRID**).
 - **Corrupted / Unreadable**: Triggers immediate **BLOCKED** status.

Phase 3: Statutory Provider & Cartographic Legend Resolution

1. ****Statutory Undertaker Taxonomy (src/warnings/catalogue.py)****:
 - Maps canonical utility names to statutory infrastructure categories:
 - **UKPN, Northern Powergrid, SP Energy Networks, NGET**: Electricity distribution and transmission.
 - **Cadent, SGN, Northern Gas Networks, Wales & West**: Gas distribution.
 - **Thames Water, Anglian Water, Severn Trent**: Clean water and foul sewerage.
 - **Openreach (BT), Virgin Media, Vodafone**: Telecommunications and fiber optics.
2. **HSG47 Warning Catalogue Lookup**:
 - Resolves active hazard thresholds: High-Voltage electrical lines (11kV, 33kV, 132kV), Low-Voltage lines (230V/400V), High-Pressure gas mains (>7 bar), Intermediate-Pressure gas (2 to 7 bar), Medium-Pressure gas (75 mbar to 2 bar), Low-Pressure gas (<75 mbar).
3. ****Cartographic Legend Resolution (src/legends/registry.py)****:
 - Loads the exact drawing legend profile:
 - **RGB / HSV Target Color Bands**: Identifies hazard line colors (e.g., Red **#FF0000** for 11kV cables, Yellow/Orange **#FFA500** for Gas, Blue **#0000FF** for Water).
 - **Delta-E Color Tolerance**: Employs CIE76 / CIE94 color distance formulas with a calibrated tolerance of $\Delta E \leq 28.0$, accommodating CAD compression and anti-aliasing variations.
 - **Stroke Width & Dash Patterns**: 0.25pt to 4.0pt line widths; solid, dashed, or dotted stroke dash arrays.

Phase 4: Spatial AOI Extraction & Geometric Normalization

1. ****Excavation Boundary Detection (src/aoi/detector.py)****:
 - The Area of Interest (AOI) represents the proposed excavation pit or trench.
 - On UK utility drawings, the AOI is typically marked as a red (**#FF0000**), magenta, or yellow dashed or solid polygon.
 - **AOIDetector** extracts red drawing paths or raster contours, filters out legends and title blocks, and assembles the perimeter vertices.
2. **CAD Viewport & Frame Detection**:
 - Discards page headers, statutory disclaimer text, and multi-column legend blocks.
 - Thames Water vector viewports: Assembles 4 boundary line primitives into an orthogonal bounding box.
 - BT raster frames: Uses morphological row/column dark-pixel scanning to identify the inner cartographic boundary.
3. ****Rotation & Coordinate Normalization (src/spatial/coordinates.py)****:

- CAD drawings often specify a visual display rotation (**/Rotate 90, 180, or 270**) while the underlying PDF mediabox remains unrotated.
- SafeDig applies transformation matrices to map unrotated PDF stream coordinates to visual screen coordinates.

4. Shapely Polygon Validation:

- Enforces topological closure (**coords[0] == coords[-1]**).
- Requires at least 4 coordinate vertices (triangle + closure) and positive non-zero area (**polygon.area > 0**).

Phase 5: Dual-Engine Spatial Asset Detection

1. **Channel A: Vector Graphics Engine (src/vector/analyzer.py)**:

- Traverses PyMuPDF drawing paths (**page.get_drawings()**).
- Filters out the AOI boundary itself so the dig-site boundary line is not mistakenly detected as an underground utility pipe!
- Compares stroke RGB color, width, and dash pattern against the resolved **LegendProfile**.
- Generates **SpatialFeature** instances with Shapely **LineString** or **MultiLineString** geometries.

2. **Channel B: Computer Vision Raster Engine (src/cv/color.py & src/cv/morphology.py)**:

- Rasterizes the drawing at **300 DPI** using PyMuPDF pixmap rendering into an OpenCV BGR numpy array.
- Converts BGR to HSV color space and applies upper/lower color threshold masks.
- Executes morphological closing and dilation (**cv2.morphologyEx**) to bridge broken line segments caused by scan compression.
- Runs **cv2.findContours** to identify point features: valves, link boxes, manholes, and chambers.

3. **Channel C: OCR Text Booster (src/ocr/service.py)**:

- Scans text blocks in and around the AOI for statutory hazard annotations: **"11kV", "33kV", "132kV", "HP GAS", "SUBSTATION", "HIGH VOLTAGE"**.
- Boosts detection confidence when spatial features coincide with voltage or pressure labels.

Phase 6: Spatial Standoff Buffering & Mathematical Reconciliation

1. **Statutory Standoff Buffer Construction (src/spatial/engine.py)**:

- Under UK HSG47, physical mechanical excavation is prohibited within safety standoff buffers around buried assets.
- SafeDig computes Shapely polygon buffers around the excavation AOI:
- **Hand-Dig Safety Buffer: 500mm** around dig perimeter.
- **High-Voltage Electrical Standoff: 3.0 meters** around 11kV/33kV cables.
- **High-Pressure Gas Exclusion Zone: 15.0 meters** around HP gas mains.

2. **2D Geometric Intersection Calculation (src/reconciliation/engine.py)**:

- Executes exact 2D spatial intersection: **intersection = buffered_aoi.intersection(asset_geometry)**.
- Computes total intersecting length (in meters/points) and minimum proximity distance to site boundary.

3. Reconciliation Outcome Classification:

- ****CONFIRMED_CLEAN****: Upstream index declared "No assets" and spatial scan confirmed 0 intersecting features.
- ****MATCH****: Upstream index declared assets present and spatial scan located matching assets in AOI.
- ****MISSED_WARNING: Upstream index declared "No assets" or omitted high-voltage warnings, but spatial scan discovered active utility lines inside the AOI! Critical safety alert.****
- ****POSSIBLE_FALSE_POSITIVE****: Marginal visual detection or ambiguous legend match.

Phase 7: The 17 Mandatory Release Gates & Policy Decision Engine

1. **Deterministic Gate Evaluation (src/policy/gates.py)**:

- Evaluates all 17 mandatory release gates sequentially.
- A single gate failure immediately prevents automated release.

2. **Priority Decision Ladder (src/policy/engine.py)**:

- **Early Exit 1**: If utility declared "No assets" and spatial engine confirms clean: **AUTO_CLEAR**.
- **Early Exit 2**: If map document missing or corrupted: **BLOCKED**.
- **Early Exit 3**: If document mapping is ambiguous (multiple plans found): **BLOCKED**.
- **Early Exit 4**: If plan provides no drawing data (portal notice only): **BLOCKED**.
- **Step 4**: If reconciliation outcome is **MISSED_WARNING: HUMAN_REVIEW** (Mandatory human escalation).
- **Step 5**: If reconciliation outcome is **POSSIBLE_FALSE_POSITIVE: HUMAN_REVIEW**.
- **Step 6**: If outcome is **MATCH** but Gate 12 fails (High-Voltage or High-Pressure Gas present): **HUMAN_REVIEW**.
- **Step 7**: If Gate 07 (Legend) or Gate 08 (AOI) failed: **HUMAN_REVIEW**.

- **Step 8 (Final Clearance):** If and only if **ALL 17 GATES PASS** and evidence is complete: **AUTO_CLEAR**.

Phase 8: LangGraph State Machine Orchestration

SafeDig organizes pipeline execution as a stateful Directed Acyclic Graph (DAG) via LangGraph ([src/orchestration/graph.py](#)):

```
(START) ■■■ ingestion_node ■■■ aoι_node ■■■ detection_node ■■■ reconciliation_node
      ■
(END) ■■■ audit_node ■■■ evidence_node ■■■ advisory_node ■■■ policy_node
```

1. ****State Dictionary ([src/orchestration/state.py](#))****:
 - **PipelineState**: Typed dictionary passing **index_records**, **documents**, **aoι**, **detections**, **reconciliation**, **policy_result**, **evidence_package**, and **audit_records** between nodes.
2. ****Error Trapping & Isolation ([src/orchestration/nodes.py](#))****:
 - If an individual document encounters a parsing exception, the error is isolated to that document record, preventing pipeline crashes and ensuring the remaining documents in the pack continue processing.

Phase 9: Advisory Local LLM Integration

1. **Zero Validation Authority Mandate**:
 - Unlike naive LLM pipelines that rely on generative AI to make safety decisions, SafeDig's LLM has **zero authority over release gates or spatial clearance**.
 - Spatial math and policy decisions are 100% deterministic.
2. ****Ollama Integration ([src/llm/client.py](#))****:
 - Asynchronously queries a local Ollama instance running **qwen2.5:7b** or **llama3.2:3b**.
 - Provides structured discrepancy explanations for human safety engineers (e.g., explaining why an undisclosed 11kV cable was flagged).
3. ****Deterministic Oms Rule Fallback ([src/llm/explainer.py](#))****:
 - If Ollama is offline, unreachable, or responds after > 3.0s, the deterministic explainer instantly constructs a standardized, grammatically precise technical summary with zero latency.

Phase 10: Forensic Evidence Packaging & Database Persistence

1. ****Evidence Package Generation ([src/evidence/crops.py](#))****:
 - **High-Resolution AOι Crop**: 300 DPI focused image of the excavation site plus 15m context margin.
 - **Whole-Sheet Overview**: Full-page overview showing the global site location.
 - **Neon Vector Overlay**: Translucent neon-highlighted overlay showing detected utility lines and buffer rings.
 - **GeoJSON Feature Set**: Exportable vector coordinates for GIS and field CAD tablets.
2. ****Forensic Database Persistence ([src/db/persistence.py](#))****:
 - Writes all job data to SQLite **safedig.db**.
 - Records SHA-256 cryptographic signatures of input PDFs, AOι geometries, gate evaluation snapshots, and timestamped audit logs.

Phase 11: Real-Time Web Console & Human Review Sign-Off

1. ****FastAPI Application ([src/api/app.py](#))****:
 - Exposes REST endpoints for job submission, status polling, and QA workspace management.
2. ****Interactive Single-Page Console ([src/api/static/](#))****:
 - **Animated Radar Scanner**: Visual radar sweep indicating active processing.
 - **HTML5 Canvas Viewer**: Hardware-accelerated smooth zoom and pan for high-resolution CAD sheets.
 - **Dynamic Gate Grid**: Interactive 17-gate status dashboard with green pass badges and red failure alerts.
 - **Human Sign-Off Modal**: Certified safety engineers can record risk acceptance or permit rejections with digital signatures stored permanently in **safedig.db**.